

Practica 5

a) //Precondicion: la cantidad de Autos, Camionetas y Camiones es la misma

Procedure Program is

Task Type Auto is

End Auto;

Task Type Camioneta is

End Camioneta;

Task Type Camion is

End Camion;

Task Puente is

Entry PedidoAuto();

Entry PedidoCamioneta();

Entry PedidoCamion();

Entry Fin(cant: IN int);

End Puente;

A: array(1..N) of Auto;

B: array(1..N) of Camioneta;

C: array(1..N) of Camion;

Task Body Auto is

Begin

Puente.PedidoAuto();

// cruzar puente

Puente.Fin(1);

End Auto;

Task Body Camioneta is

Begin

```
Puente.PedidoCamioneta();
```

```
// cruzar puente
```

```
Puente.Fin(2);
```

End Camioneta;

Task Body Camion is

Begin

```
Puente.PedidoCamion();
```

```
// cruzar puente
```

```
Puente.Fin(3);
```

End Camion;

Task Body Puente is

```
i, unidades: int;
```

Begin

```
unidades = 0;
```

```
while true{
```

```
  SELECT
```

```
    accept Fin(cant: IN int)
```

```
    unidades -= cant;
```

```
  OR
```

```
    when (unidades + 1 <= 5) =>
```

```
      accept PedidoAuto();
```

```
      unidades++;
```

```
  OR
```

```
    when (unidades + 2 <= 5) =>
```

```
      accept PedidoCamioneta();
```

```
      unidades += 2;
```

```
  OR
```

```
    when (unidades + 3 <= 5) =>
```

```
      accept PedidoCamion();
```

```
      unidades += 3;
```

```
END SELECT;
```

```

    }
End Puente;

Begin

End Program;

```

b) //Precondicion: la cantidad de Autos, Camionetas y Camiones es la misma

Procedure Program is

Task Type Auto;

Task Type Camioneta;

Task Type Camion;

Task Puente is

```

    Entry PedidoAuto();
    Entry PedidoCamioneta();
    Entry PedidoCamion();
    Entry Fin();

```

End Puente;

A: array(1..N) of Auto;

B: array(1..N) of Camioneta;

C: array(1..N) of Camion;

Task Body Auto is

Begin

```

    Puente.PedidoAuto();
    // cruzar puente
    Puente.Fin(1);

```

End Auto;

Task Body Camioneta is

Begin

```

    Puente.PedidoCamioneta();
    // cruzar puente
    Puente.Fin(2);
End Camioneta;

Task Body Camion is
Begin
    Puente.PedidoCamion();
    // cruzar puente
    Puente.Fin(3);
End Camion;

Task Body Puente is
    i, unidades: int;
Begin
    unidades = 0;
    while true{
        SELECT
            accept Fin(cant: IN int)
            unidades -= cant;
        OR
            when ((PedidoCamion'count = 0) AND (unidades + 1 <= 5)) ⇒
            accept PedidoAuto() do
                unidades++;
            end PedidoAuto;
        OR
            when ((PedidoCamion'count = 0) AND (unidades + 2 <= 5)) ⇒
            accept PedidoCamioneta() do
                unidades += 2;
            end PedidoCamioneta;
        OR
            when (unidades + 3 <= 5) ⇒
            accept PedidoCamion() do
                unidades += 3;
            end PedidoCamion;
        END SELECT;
    }
End Puente;

```

Begin

End Program;

Ejercicio 2

a_

```
procedure ejercicio_2_a
task type Cliente;

task Empleado is
begin
    Entry atenderCliente(pago: IN string; comprobante: OUT string);
end Empleado;

c: array[c] of Cliente;

task body Cliente is
comprobante: string;
begin
    Empleado.atenderCliente(realizarPago(), comprobante)
end Cliente;

task body Empleado is
cant: int;
begin
    for (cant = 0 to C-1){
        accept atenderCliente(pago: IN string; comprobante: OUT string) do
            comprobante = realizarComprobante();
        end atenderCliente;
    }
end Empleado;

begin
```

```
end ejercicio_2_a;
```

b_

```
procedure ejercicio_2_b
task type Cliente;

task Empleado is
begin
    Entry atenderCliente(pago: IN string; comprobante: OUT string);
end Empleado;

c: array[c] of Cliente;

task body Cliente is
comprobante: string;
begin
    SELECT
        Empleado.atenderCliente(realizarPago(), comprobante)
    OR DELAY 600
        null;
    END SELECT; ****
end Cliente;

task body Empleado is
cant: int;
begin
    while (true){
        accept atenderCliente(pago: IN string; comprobante: OUT string) do
            comprobante = realizarComprobante();
        end atenderCliente;
    }
end Empleado;

begin

end ejercicio_2_b;
```

c_

```
procedure ejercicio_2_c
task type Cliente;

task Empleado is
begin
    Entry atenderCliente(pago: IN string; comprobante: OUT string);
end Empleado;

c: array[c] of Cliente;

task body Cliente is
comprobante: string;
begin
    SELECT
        Empleado.atenderCliente(realizarPago(), comprobante)
    ELSE
        null;
    END SELECT; ****
end Cliente;

task body Empleado is
cant: int;
begin
    while (true){
        accept atenderCliente(pago: IN string; comprobante: OUT string) do
            comprobante = realizarComprobante();
        end atenderCliente;
    }
end Empleado;

begin

end ejercicio_2_c;
```

d_

```

procedure ejercicio_2_b
task type Cliente;

task Empleado is
begin
    Entry atenderCliente(pago: IN string; comprobante: OUT string);
end Empleado;

c: array[c] of Cliente;

task body Cliente is
comprobante: string;
begin
    SELECT
        Empleado.atenderCliente(realizarPago(), comprobante)
    OR DELAY 600
    SELECT
        Empleado.atenderCliente(realizarPago(), comprobante)
    ELSE
        null;
    END SELECT;
    END SELECT; ****
end Cliente;

task body Empleado is
cant: int;
begin
    while (true){
        accept atenderCliente(pago: IN string; comprobante: OUT string) do
            comprobante = realizarComprobante();
        end atenderCliente;
    }
end Empleado;

begin

end ejercicio_2_b;

```


Ejercicio 3

```
procedure Sistema is
Task Central is
    Entry recibirSenial1();
    Entry recibirSenial2();
    Entry finPrioridad();
End Central;

Task Timer is
    Entry comenzar();
End Timer;

Task Periferico1;
Task Periferico2;

Task Body Central is
    bool prioridadPeriferico2 = false;
begin
    accept recibirSenial1();
LOOP
    if (!prioridadPeriferico2){
        SELECT
            accept recibirSenial1();
        OR
            accept recibirSenial2() do
                prioridadPeriferico2 = true;
                Timer.comenzar();
            End recibirSenial2;
        END SELECT;
    } else {
        SELECT
            accept finPrioridad() do
                prioridadPeriferico2 = false;
            End finPrioridad;
        OR
            accept recibirSenial2();
        END SELECT;
    }
end;
```

```

    }
    END LOOP;
End Central;

Task Body Timer is
begin
    LOOP
        accept comenzar();
        delay 180;
        Central.finPrioridad();
    END LOOP;
End Timer;

Task Periferico1 is
begin
    LOOP
        SELECT
            Central.recibirSenial1();
        OR DELAY 120
            null;
        END SELECT;
    END LOOP;
end Periferico;

Task Periferico2 is
begin
    LOOP
        SELECT
            Central.recibirSenial2();
        OR DELAY 60
            delay 1;
        END SELECT;
    END LOOP;
end Periferico;

Begin

End Sistema;

```

Ejercicio 4

Ejercicio 5

```
procedure Sistema is {

    task Servidor is
        ENTRY atenderPedido(Doc: IN txt, Res: OUT boolean);
    end Servidor;

    task type Usuario;

    vectorUsuarios: array[U] of Usuario;

    task body Servidor is
    begin
        loop
            accept atenderPedido(Doc: IN txt, Res: OUT boolean) do
                Res = analizarDocumento(Doc)
            end atenderPedido;
        end loop;
    end Servidor;

    task body Usuario is
        boolean listo;
        txt doc;
    begin
        listo = false
        TrabajarDoc(doc)
        while (not listo) loop
            select
                Servidor.atenderPedido(doc, listo)
            if (not listo)
```

```

        TrabajarDoc(doc)
    or delay (120)
        delay(60)
    end loop
end Usuario;
end Sistema;

```

Ejercicio 6

```

procedure Juego is
task type Persona is
    entry recibirGanador(equipo: in integer);
end Persona;

task type Equipo is
    entry ident(id: in integer);
    entry llegada;
    entry iniciarJuego;
    entry termineJuego(monedas: in integer);
end Equipo;

task Administrador is
    entry enviarTotal(total: in integer, equipo: in integer);
end Administrador;

vectorPersonas: array[1..20] of Persona;
vectorEquipo: array[1..5] of Equipo;

task body Equipo is
    idEquipo: integer;
    total: integer;
    accept Ident (id: in Integer) di
        idEquipo:= id;
    end Ident;
    for i in 1..4 loop
        accept llegada;

```

```

end loop;
for i in 1..4 loop
    accept iniciarJuego;
end loop;
total := 0;
for i in 1..4 loop
    accept termineJuego(monedas: in integer) do
        total += monedas;
    end;
end loop

Administrador.enviarTotal(monedas, idEquipo);

end Equipo;

task body Persona
    equipo: integer;
    moneda: integer;
    ganador: integer;
begin
    moneda := 0;
    Equipo(equipo).llegada
    Equipo(equipo).iniciarJuego

    for i in 1..15 loop
        monedas += Moneda();
    end loop;

    Equipo(equipo).termineJuego(monedas);

    Administrador.recibirGanador(ganador);
end

task body Administrador
    int max;
    int maxEquipo;
    int equipoGanador;
begin

```

```

max := -99999;
for i in 1..5 loop
    accept enviarTotal(monedas: in integer, equipo: in integer) do
        if (monedas > max)
            max := monedas;
            maxEquipo := equipo;
        end if;
    end enviarTotal;
end loop

for i in 1..20 loop
    accept recibirGanador(ganador: out integer) do
        ganador := maxEquipo;
    end recibirGanador
end loop;
end

Begin
    for i in 1..5 loop
        vectorEquipo(i).Ident(i);
    end loop;
End Juego;

```

Ejercicio 7

```

procedure valorPromedio is

task Coordinador is
    entry recibirPorcion(porcion: IN integer)
end;

task type Worker;
vectorWorker: array[1..10] of Worker;

task body Coordinador is
    total integer := 0;

```

```

    promedio: float;
begin
    for i in 1..10 loop
        accept recibirPorcion(porcion: IN integer) do
            total += porcion;
        end recibirPorcion;
    end loop;
    integer promedio = total/1000000;
end Coordinador;

task body Worker is
    vectorValores: array[1..100000] of integer:= inicializarVector;
    subtotal: integer:= 0;
begin
    for i in 1..100000 loop
        subtotal += vectorValores[i];
    end loop;
    Coordinador.recibirPorcion(subtotal);
end Worker;

end;

begin

end;

```

Ejercicio 8

```

procedure HuellasDactilares is

task Especialista is
    entry ResultadoMuestra(codigo: IN integer, similitud: IN integer);
end Especialista

task Coordinador is
    entry recibirlImagen(test: in Test);

```

```

    entry recibirResultado(codigo: IN integer, similitud: IN integer)
end Coordinador

task type Servidor is
    entry recibirlImagen(test: in Test);
end Servidor
vectorServidores: array[1..8] of Servidor;

task body Especialista is
    test: Test;
    codigo: integer;
    similitud: integer;
    maxCodigo: integer
    maxSimilitud: integer:= -1;
begin
    loop
        tomarImagen(test);
        Coordinador.recibirlImagen(test);
        for i in 1..8 loop
            accept ResultadoMuestra(codigo: IN integer, similitud: IN integer) do
                if (similitud > maxSimilitud)
                    maxSimilitud = similitud;
                    maxCodigo = codigo;
                end if;
            end ResultadoMuestra;
        end loop
    end loop;
end Especialista

task body Coordinador is
begin
    loop
        select
            accept recibirlImagen(test: In test) do
                for i in 1..8 loop
                    Servidor(i).recibirlImagen(test);
                end RecibirlImagen;
            or

```



```

        accept recibirResultado(codigo: IN integer, similitud: IN integer) do
            Especialista.ResultadoMuestra(codigo, similitud);
        end loop;
    end Coordinador

task body Servidor is
    codigo, valor: integer;
begin
    loop
        accept RecibirImagen(test: In test) do
            Buscar(test, código, valor);
            Coordinador.recibirResultado(código, valor);
        end loop
    end Servidor;

begin
end;

```

Refactorizado

```

procedure HuellasDactilares is

task Especialista is
    entry ResultadoMuestra(codigo: IN integer, similitud: IN integer);
end Especialista

task type Servidor is
    entry recibirImagen(test: in Test);
end Servidor
vectorServidores: array[1..8] of Servidor;

task body Especialista is
    test: Test;
    codigo: integer;
    similitud: integer;
    maxCodigo: integer

```

```

    maxSimilitud: integer:= -1;
begin
    loop
        tomarImagen(test);
        for i in 1..16 loop
            SELECT
                accept RecibirImagen(imagen: out test) do
                    imagen = test;
                end RecibirImagen;
            OR
                accept ResultadoMuestra(codigo: IN integer, similitud: IN integer)
do
                if (similitud > maxSimilitud)
                    maxSimilitud = similitud;
                    maxCodigo = codigo;
                end if;
            end ResultadoMuestra;
        end loop
    end loop;
end Especialista

task body Servidor is
codigo, valor: integer;
begin
    loop
        Especialista.recibirImagen(test);
        Buscar(test, código, valor);
        Especialista.recibirResultado(código, valor);
    end loop
end Servidor;

begin
end;

```

Ejercicio 9

```

procedure Recoleccion is

task type Camion is
end Camion;

task type Persona is
    entry Ident (idPersona: in Integer);
end Persona;

task Coordinador is
    entry recibirReclamo(res: out boolean, id: in integer, cantReclamos: in integer);
end Coordinador;

vectorPersonas: array[1..p] of Persona;
vectorCamiones: array[1..3] of Camion;

task body Camion is
end Camion;

task body Persona is
    listo: boolean;
    id: integer;
    cantReclamos: integer:= 0;
begin
    accept Ident (idPersona: in Integer) do
        id:= idPersona;
    end Ident;
    listo:= false;
    while (not listo)
        cantReclamos++;
    select
        Coordinador.recibirReclamo(listo, id, cantReclamos);
    or delay (900)
        NULL
    end select;
end Persona;

task body Coordinador is

```

```

camiones: integer;
begin
  camiones := 0;
  loop
    select
      accept recibirReclamo(res: out boolean, id: in integer, cantReclamo
s: in integer) do
        if (camiones == 0)
          res := false;
          vector[id] = cantReclamos;
        else
          res = true;
          camiones--;
        end recibirReclamo;
      or
        accept camionListo
          camiones++;
        end camionListo;
    end loop
  end Coordinador;

Begin
  for i in 1..p loop
    vectorPersonas(i).Ident(i);
  end loop;
end recoleccion;

```

```

SELECT
  accept camionesListo() do
    if (not cola.isEmpty())
      cola.remove(idAux, cantReclamosAux);
      Persona(idAux).enviarCamion;
    else
      camiones++;
    end if;
  OR

```

```
when (camiones = 0) OR (not cola.isEmpty)
```

```
  accept recibirReclamo() do
```

```
    cola.add(id, cantReclamos)
```

```
  end recibirReclamo;
```

```
OR
```

```
  when (cola.isEmpty() AND camiones > 0)
```

```
    accept recibirReclamo(idAux, cantReclamosAux) do
```

```
      camiones--;
```

```
      Persona(idAux).enviarCamion;
```

```
OR
```

```
  accept quitarReclamo(idAux, cantReclamoAux) do
```

```
    cola.remove(idAux, cantReclamoAux)
```

```
  end quitarReclamo;
```